

Description of the SDK for the transfer of legal texts

Note: *This document describes the PHP SDK. For a description of the XML exchange format, please read the document LTI XML Specification.

The data is transferred to your interface via POST in XML format. The XML document is UTF-8 encoded. Currently, the XML document can be transferred as `Content-Type: text/xml` (preferred) or alternatively as `application/x-www-form-urlencoded` with the POST parameter `xml ($_POST['xml'])`. Other agreements are possible.

1 Implementation and use of the PHP SDK

We offer a PHP SDK for users of our IT-Recht Kanzlei interface. The SDK is designed for easy use. First, the two abstract methods of the class `LTIHandler` class and one of the two authentication methods must be overwritten.

1.1 Creation of your own LTI handler

1.1.1 `preHandleRequest()`

This method can be used to initialize resources or to validate preconditions (e.g. configuration settings) that the target system has to fulfill in order to operate.

If the required conditions are not met, you can throw an exception of type `LTIError` here, which will be converted to a properly formatted error response.

1.1.2 Authentication methods

To ensure that the transfer of the legal texts originates from the IT-Recht Kanzlei system, you can implement one of the following two methods.

`isTokenValid(string $token): bool`

The validity of the transmitted token must be checked here if the system is to work with a token.

The token should be generated automatically by your system. To generate a random token, you can use the `LTI::generateToken()` method. The token should be displayed to the user of your interface so that the user can store the token in the client portal of the IT-Recht Kanzlei.

This is the preferred authentication method.

`validateUserPass(string $username, string $password): bool`

The correctness of a transmitted user name and password must be checked here. The username and password can be assigned by the users themselves in your plugin or can be created by your plugin. The username and password must then be stored in the client portal when setting up the interface.

Please inform the technical support of IT-Recht Kanzlei in advance whether the password should be transmitted in plain text or hashed (incl. algorithm).

1.1.3 `handleActionGetAccountList(): LTIAccountListResult`

This function should return a list of all sales channels in your system. An ID (`accountid`) and the name (`accountname`) must be specified for each sales channel. In addition, a list of available target languages and countries can be transferred for each sales channel.

Even if your system is not a multishop system, it is recommended to implement this function to announce the supported target languages and countries of your system. For this use case, only one sales channel needs to be specified where the ID is specified as 0. The account name can remain empty.

1.1.4 `handleActionPush(LTIPushData $data): LTIPushResult`

Once the request has been received by the plugin, the document must be integrated into the corresponding target page. The document is processed here. The `LTIPushData` object has getter methods for all properties of the document that can be queried and subsequently processed.

Please refer to the Doc-Comments provided in the `LTIPushData` class for further information.

If your system is unable to process the legal text, for example because the document is written in a language that your system does not support, please throw an exception of type `LTIError` with the corresponding error code and an error message that allows the user of your interface to correct the error themselves. For more details, please read the section “Error Codes”.

1.2 Use of the LTI class and the LTI handler you have implemented.

The class `LTI` (`LTI.php`) takes over the preprocessing of the incoming XML data, evaluates the action to be executed and carries out most of the error handling. This file should not be changed to ensure that the SDK remains fully functional.

The constructor of the `LTI` class requires three parameters.

1. An instance of your overwritten `LTIHandler` class
2. The version of the system that is addressed with your implementation
3. The version of your implementation in which the SDK is used

The last step is to call the `handleRequest()` method of the `LTI` class object. The received XML document is passed on as a string as the parameter.

Both error handling within the SDK and the preparation of the response for the IT-Recht Kanzlei server are handled automatically by the SDK. The developer can ignore these aspects.

An example implementation for using the SDK can be found in the directory `example-implementation`.

2 General

2.1 Versioning

Requirements for the processing of version numbers by the client portal of IT-Recht Kanzlei:

- `meta_modulversion`: Assign incrementing version numbers for your implementation when you first create it and for each subsequent update.
- `meta_shopversion`: If possible, please also transmit a system version number that can be compared using comparison operators (e.g. “2.0” instead of “XY2” - otherwise by request). Please let us know the structure of your versioning scheme (e.g. “major.minor”) so that we can process it correctly internally.

2.2 Multishop systems

If your system is a so-called multishop system, i.e. several sales channels/services exist under one administration interface, it is necessary that the user of your implementation is offered a list of available sales channels/services for which the legal texts are to be transferred in the client portal of the IT-Recht Kanzlei.

The list of sales channels is retrieved using the `handleActionGetAccountList()` function.

During the subsequent transfer of a legal text, the ID of the sales channel selected by the user can be retrieved within the `handleActionPush()` method for multishop systems from the `LTIPushData` object using the `getMultiShopId()` method.

2.3 Best Practices

- Assign proper numerical, incrementing version numbers (e.g. “1.0”, “1.1”, “1.2”, ...) for your module when it is first created and for each subsequent update. In addition to classic version numbers, the release date can also be used numerically here (e.g. “20230827”, format YYYYMMDD). Always state the current version number in the documentation / installation instructions supplied with the module and at least in the main program file.
- Include easy-to-understand installation instructions with your module download or on the download page. Include special cases in your description (e.g. for older store versions).
- Include a description for the user of new versions of your implementation (updates) on how to update to the latest version (this often differs from the classic installation instructions), unless the update runs automatically (e.g. by clicking on “Update” in the store’s module store).
- Include a changelog.txt or similar with new releases of your implementation (updates) to inform the user about the new features and bug fixes.

2.4 Status Codes

- **success:** Everything went well. You confirm,
 - that no errors occurred when querying the version
 - that no errors occurred when querying the account list
 - that when a legal text was pushed, it was successfully published in the user’s account/shop
- **error:** Regardless of the error code, the status of the error response will always be “error”.

2.5 Error Codes

The error codes are documented in the `LTIErrors.php` file. If possible, please only use the error codes defined in the file.

Avoid using the error code 99 as much as possible.

You can define your own error codes with the number range ≥ 100 . Please inform IT-Recht Kanzlei of the error code and its meaning. Error codes for other generic errors can also be added to the number range < 100 by agreement.

3 Testing Tools

To test your implementation of the interface, please read the `README.md` in the `testSuite` directory.

You can use the LTI Test Tool as a further tool for testing. There you can enter the API URL and the authentication data for your test system and execute requests with the legal text interface of the IT law firm.

4 Integration of your system into the client portal

As soon as you have completed the integration into your system, please contact the technical support of IT-Recht Kanzlei. The following information is required to integrate your system into the IT-Recht Kanzlei's client portal:

- Endpoint at which your connection is accessible,
- Type of data transfer (form-urlencoded or as XML document),
- Authentication method (token or username/password and password hash algorithm, if applicable)
- Does your system support PDF attachments

If possible, please provide the IT-Recht Kanzlei technicians with test access to your system. This enables the creation of instructions for future users of the interface and a final quality control.

5 Details of the implementation within the SDK

If you are unable to use the SDK, you will find a complete description of the XML requests and responses, including some examples, in the document LTI XML Specification.

Further examples can be found in the directories `example-xmIs` and `testCases`.